

PROJECT PROPOSAL

System Call Implementation

Group # 36

Group member 1 – Jaikumar Wath

Group name – KernelX

Group member 2 – Ayush Pareek

Scheduler History System Call in Linux

Objective

The objective of this project is to design and implement a custom Linux system call that exposes the scheduling history of a process over time. The system call will allow user-space programs to retrieve detailed information about when a process is scheduled in and scheduled out by the kernel, along with the CPU core on which it executes.

Motivation

In operating systems theory, scheduling algorithms such as First-Come First-Serve, Round Robin, and Priority Scheduling are studied in isolation. However, modern operating systems like Linux use highly dynamic, preemptive, and priority-based schedulers that behave differently in real systems. Existing user-space tools provide only instantaneous or aggregated scheduling information and do not reveal the actual sequence of scheduling events. This project aims to bridge this gap by making real scheduler behavior observable at the kernel level.

Proposed Solution

We propose implementing a new Linux system call, tentatively named `sys_get_sched_history(pid_t pid, struct sched_event __user *buf, int max_events)`, which returns a recent history of scheduling events for a specified process. The system call records scheduling-in and scheduling-out events inside the kernel using a fixed-size ring buffer to ensure bounded memory usage.

The system call accepts a process ID to identify the target process, a user-space buffer pointer into which scheduling records are safely copied, and a maximum event count that limits the amount of data transferred to user space. Each scheduling record is represented using a `sched_event` structure containing the event timestamp, the CPU core on which the process executed, and the type of scheduling event.

Scope of the Project

Included:

- Recording scheduling-in and scheduling-out events
- Multi-core (SMP) scheduling awareness
- Kernel timekeeping
- Safe user-kernel data transfer

Excluded:

- Modifying scheduler behavior or policies
- Memory management or paging
- File system or I/O operations
- Persistent kernel logging

Expected Outcome

The system call will allow user-space programs to retrieve and display the scheduling history of a process, including when it is scheduled in and out and the CPU core on which it executes. This enables practical observation of preemptions and CPU migrations in a multi-core Linux system.

A sample user-space output may appear as follows:

Time (ms)	CPU	Event
12.5	1	SCHEDULED_IN
14.2	1	SCHEDULED_OUT
15.0	3	SCHEDULED_IN